

Pet Care AI Microservice

Server Performance Evaluation

Measured 2026-09-23 | code at commit f567cdc | branch perf/onnx-tokenizer-padding

1. Summary

The FastAPI service exposes symptom classification (**/predict**), stateless triage chat (**/chat**), rule-based wellness scoring (**/wellness**) and a deterministic feeding summary (**/feeding-summary**). Conditions are predicted by a fine-tuned DistilBERT classifier (16 classes); Google Gemini only writes human-facing prose. This report measures request latency and throughput of the service itself (Gemini disabled), the end-to-end latency with live Gemini, and model cold start.

Key findings:

- **Performance defect found and fixed.** The production ONNX backend padded every input to 256 tokens (real inputs are ~16). Removing the padding reduced **/predict** median latency from 62.5 ms to 5.8 ms (x10.8) and raised peak throughput from 20.5 to 255.7 requests/s (x12.5), with unchanged classification accuracy.
- **Service overhead is small.** Non-model endpoints answer in under 3 ms; after the fix, model-backed endpoints answer in 5-9 ms at the median.
- **Gemini dominates user-perceived latency.** With live Gemini, responses take 1.0-1.6 s at the median, of which the service's own processing is a few percent. The deterministic emergency path (no LLM call) stays well under 100 ms.
- **Cold start is fast.** Loading the quantized ONNX model takes 180 ms (median of 5 fresh processes).

2. Test Environment and Methodology

Parameter	Value
Hardware	Apple M2, 8 CPU cores (4 performance + 4 efficiency), 16 GB RAM
OS / runtime	macOS (Darwin 24.6), Python 3.11
Server	uvicorn, 1 worker process (sync endpoints run in its thread pool)
ML backends	ONNX Runtime 1.19.2, int8-quantized DistilBERT (production); PyTorch 2.4.1 fp32 (reference)
Load generator	scripts/benchmark_http.py: asyncio + httpx, same machine, closed-loop workers
Sample size	200 requests per endpoint per concurrency level (1, 4, 16); 5 warm-up requests excluded
Gemini	Disabled for service-only runs (local template fallback); gemini-2.5-flash-lite for end-to-end run
Metrics	Client-side wall time per request: mean, p50, p95, p99, max; throughput = completed requests / wall time

Table 1. Test environment.

Concurrency c is the number of simultaneous clients, each sending its next request as soon as the previous one completes. Latencies are reported as **p50 / p95** in milliseconds.

3. Results

3.1 Endpoints without the ML model

These endpoints are deterministic (no classifier, no LLM) and backend-independent.

Endpoint	p50 / p95 (ms)	Peak RPS
GET /health	0.9 / 1.0	1923
POST /wellness	2.1 / 2.4	682
POST /feeding-summary (100 pets)	2.6 / 2.8	625

Table 2. Latency at c = 1 and peak throughput over c = 1, 4, 16.

3.2 Model-backed endpoints by inference backend

Production (AWS Lambda) runs the int8 ONNX model on CPU. PyTorch results are given for reference; locally PyTorch picks the Apple GPU (MPS), so it was also measured with the GPU hidden to match Lambda's CPU-only environment. The PyTorch and ONNX bundles are different model builds, so only their speed (not accuracy) is compared.

Endpoint	ONNX before fix	ONNX after fix	PyTorch CPU	PyTorch GPU (MPS)
POST /predict	62.5 / 94.4	5.8 / 11.7	20.2 / 28.0	11.3 / 12.7
POST /predict (red flag)	66.1 / 103.4	5.0 / 10.2	25.5 / 32.2	15.0 / 16.7
POST /chat (health, 3 turns)	68.1 / 101.9	8.6 / 14.7	22.8 / 28.1	13.6 / 15.0
POST /chat (general)	69.1 / 104.3	5.5 / 9.3	22.9 / 30.0	15.0 / 16.4
Peak RPS, /predict	20.5	255.7	64.2	152.9

Table 3. p50 / p95 latency (ms) at c = 1, Gemini disabled.

3.3 ONNX backend before and after the padding fix

Endpoint	c	Before p50 / p95	After p50 / p95	Speed-up	RPS before	RPS after
POST /predict	1	62.5 / 94.4	5.8 / 11.7	x10.8	15.0	156.6
	4	201.3 / 229.5	16.0 / 27.4	x12.6	19.6	238.2
	16	767.3 / 890.3	35.0 / 182.3	x21.9	20.5	255.7
POST /predict (red flag)	1	66.1 / 103.4	5.0 / 10.2	x13.2	14.3	177.9
	4	208.6 / 243.6	14.0 / 19.4	x14.9	18.9	279.2
	16	811.7 / 930.0	31.8 / 222.3	x25.5	19.6	248.5
POST /chat (health, 3 turns)	1	68.1 / 101.9	8.6 / 14.7	x7.9	13.9	104.1
	4	217.6 / 244.7	24.0 / 30.0	x9.1	18.2	169.5
	16	838.8 / 954.1	72.3 / 213.7	x11.6	18.9	172.7
POST /chat (general)	1	69.1 / 104.3	5.5 / 9.3	x12.6	13.5	166.0
	4	222.7 / 256.7	15.2 / 20.4	x14.7	17.8	260.5
	16	958.6 / 1455.1	34.0 / 193.6	x28.2	15.5	237.2

Table 4. ONNX backend under increasing concurrency, Gemini disabled.

At c = 16 a single uvicorn process on 8 cores saturates, so latency grows roughly linearly with queue depth; throughput is the more meaningful figure there.

3.4 Classifier inference and accuracy

The classifier was also timed in-process (no HTTP) over the 496-example owner-language hold-out set, and its accuracy was compared before and after the fix.

Metric	Before fix	After fix
Tokens per input (median / max)	256 / 256	16 / 25
Inference latency p50 / p95 (ms)	65.3 / 106.7	4.8 / 10.1
Top-1 accuracy	0.679	0.677
Top-3 accuracy	0.875	0.881

Table 5. ONNX classifier on data/owner_eval.parquet (n = 496).

Top-1 predictions differ on 24 of 496 inputs because dynamic int8 quantization computes activation scales over the whole tensor, padding included; aggregate accuracy is unchanged within noise.

3.5 End-to-end latency with live Gemini

10 sequential requests per endpoint (c = 1) against the production code path before the fix, with gemini-2.5-flash-lite. All responses were generated by Gemini (no fallbacks); two requests passed through API-key rotation after a quota (HTTP 429) response, which is included in their latency.

Endpoint	p50 / p95 (ms)	Mean (ms)	Gemini call
POST /predict	1554.2 / 1740.3	1560	yes
POST /chat (general)	1546.0 / 1804.4	1514	yes
POST /chat (health)	1175.0 / 1639.6	1245	yes
POST /wellness	995.2 / 1178.1	1035	yes
POST /predict (red flag)	62.4 / 87.3	67	no (safety short-circuit)

Table 6. End-to-end latency including the Gemini call.

The fix removes roughly 55-65 ms of classifier time from each classifier-backed request, i.e. about 4-5% of end-to-end latency when Gemini is involved (estimated from Tables 3 and 6, not re-measured).

3.6 Cold start

ONNX model construction (OnnxPredictor.from_paths) in a fresh Python process **median 180 ms**, max 196 ms (samples: 194, 179, 180, 196, 170 ms). For comparison, importing and loading the fp32 PyTorch model takes about 550 ms (single run). Lambda container image pull and runtime initialisation are not included.

4. Optimization: Tokenizer Padding

Cause. The tokenizer.json exported with the model stores a fixed padding strategy of 256 tokens, left over from training. The PyTorch path uses the Hugging Face tokenizer, which overrides padding per call, but the ONNX path loads the raw tokenizers.Tokenizer, which applies the stored setting. Every request therefore ran the transformer over 256 positions, and self-attention cost grows quadratically with sequence length.

Fix. One line in OnnxPredictor.from_paths (app/inference/onnx_predictor.py):
tokenizer.no_padding() after loading. Truncation at 256 tokens is kept. A regression test (tests/test_onnx_padding.py) builds a tokenizer with fixed padding and asserts the model receives unpadded input; it fails without the fix.

Effect. 8-13x lower median latency on classifier-backed endpoints, 9-15x higher throughput, equal accuracy. On AWS Lambda (about one vCPU at 1769 MB) the absolute saving per request is expected to be larger, and billed duration drops for every classifier-backed request.

5. Limitations

- The load generator ran on the same machine as the server, so at $c = 16$ client and server compete for CPU (visible as rising p95 even on /health).
- Results are from a local Apple M2, not from the deployed Lambda (x86_64, about one vCPU); absolute numbers there will differ.
- The Gemini sample is small (10 requests per endpoint) to preserve API quota, and its latency depends on network conditions and Google's service load.
- Accuracy was checked on the owner-language hold-out, which covers 5 of the 16 classes.

6. Reproduction

```
GEMINI_API_KEY= GEMINI_API_KEYS= API_KEY= MODEL_BACKEND=onnx \  
  MODEL_PATH=models/transformer_model_onnx uvicorn app.main:app --port 8000  
python scripts/benchmark_http.py --requests 200 --concurrency 1,4,16 \  
  --output docs/benchmarks/<run>.json
```

Raw results for every table are in docs/benchmarks/*-2026-09-23.json.